Hadoop MapReduce Matrix Multiplication

1. Introduction

This program performs **matrix multiplication using Hadoop MapReduce**.

The program consists of three main parts:

1. Mapper
2. Reducer
3. Driver program

For example:

Matrix A:

```
1  2
3  4
```

Matrix B:

```
5  6
7  8
```

The multiplication result is:

```
19  22
43  50
```

---

2. Complete Java Program

Save the following program as:

```
MatrixMultiplication.java
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;
```

```
import java.util.HashMap;
import java.util.Map;

public class MatrixMultiplication {

    // Mapper Class
    public static class MatrixMapper
            extends Mapper<LongWritable, Text, Text, Text> {

        @Override
        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            Configuration conf = context.getConfiguration();

            int m = Integer.parseInt(conf.get("m")); // rows in A
            int p = Integer.parseInt(conf.get("p")); // columns in B

            String line = value.toString();
            String[] parts = line.split(",");

            String matrix = parts[0]; // A or B
            int i = Integer.parseInt(parts[1]); // row
            int j = Integer.parseInt(parts[2]); // column
            int val = Integer.parseInt(parts[3]); // value

            if (matrix.equals("A")) {

                // For each element A[i][j],
                // emit for all columns of B
                for (int k = 0; k < p; k++) {
                    context.write(
                        new Text(i + "," + k),
                        new Text("A," + j + "," + val)
                    );
                }

            } else {

                // Matrix B
                // For each element B[i][j],
                // emit for all rows of A
                for (int k = 0; k < m; k++) {
                    context.write(
                        new Text(k + "," + j),
                        new Text("B," + i + "," + val)
                    );
                }
            }
        }
    }

    // Reducer Class
    public static class MatrixReducer
            extends Reducer<Text, Text, Text, Text> {

        @Override
```

```
    public void reduce(Text key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {

        Map<Integer, Integer> aElements = new HashMap<>();
        Map<Integer, Integer> bElements = new HashMap<>();

        // Separate A and B elements
        for (Text val : values) {

            String[] parts = val.toString().split(",");

            String matrix = parts[0];
            int index = Integer.parseInt(parts[1]);
            int value = Integer.parseInt(parts[2]);

            if (matrix.equals("A")) {
                aElements.put(index, value);
            } else {
                bElements.put(index, value);
            }
        }

        // Calculate dot product
        int sum = 0;

        for (Integer k : aElements.keySet()) {

            if (bElements.containsKey(k)) {
                sum += aElements.get(k) * bElements.get(k);
            }
        }

        context.write(key, new Text(String.valueOf(sum)));
    }
}

// Driver Code
public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();

    // Matrix dimensions
    conf.set("m", "2"); // rows in A
    conf.set("p", "2"); // columns in B

    Job job = Job.getInstance(conf, "Matrix Multiplication");

    job.setJarByClass(MatrixMultiplication.class);

    job.setMapperClass(MatrixMapper.class);
    job.setReducerClass(MatrixReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

```
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

---

3. Required Hadoop JARs

The program uses Hadoop MapReduce libraries.

The important Hadoop libraries include:

```
hadoop-common-*.jar
hadoop-mapreduce-client-core-*.jar
hadoop-hdfs-client-*.jar
hadoop-auth-*.jar
```

There are also dependency JARs inside:

```
$HADOOP_HOME/share/hadoop/common/lib/
```

and:

```
$HADOOP_HOME/share/hadoop/mapreduce/
```

It is recommended **not to manually add every JAR**.

If Hadoop is installed correctly, use:

```
hadoop classpath
```

This automatically provides the required Hadoop classpath.

---

4. Check Hadoop Installation

First check Java:

```
java -version
```

Check Java compiler:

```
javac -version
```

Check Hadoop:

```
hadoop version
```

Check Hadoop classpath:

```
hadoop classpath
```

If these commands work, Hadoop is installed and available in the PATH.

---

5. Create the Java File

Create a file named:

```
MatrixMultiplication.java
```

Paste the complete Java program into this file.

The filename must match the public class:

```
public class MatrixMultiplication
```

---

6. Compile the Java Program

Run:

```
javac -classpath "$(hadoop classpath)" -d . MatrixMultiplication.java
```

After successful compilation, class files will be generated.

For example:

```
MatrixMultiplication.class
MatrixMultiplication$MatrixMapper.class
MatrixMultiplication$MatrixReducer.class
```

---

7. Create the JAR File

Create the Hadoop JAR file:

```
jar -cvf MatrixMultiplication.jar *.class
```

This creates:

```
MatrixMultiplication.jar
```

---

8. Create the Input File

Create a file:

```
matrix_input.txt
```

Put the following data inside:

```
A,0,0,1
A,0,1,2
A,1,0,3
A,1,1,4
B,0,0,5
B,0,1,6
B,1,0,7
B,1,1,8
```

The input represents the following matrices.

Matrix A:

```
1  2
3  4
```

Matrix B:

```
5  6
7  8
```

---

9. Start Hadoop

If using a pseudo-distributed Hadoop installation, start HDFS:

```
start-dfs.sh
```

Start YARN:

```
start-yarn.sh
```

Check the running Hadoop processes:

```
jps
```

Normally, you should see processes such as:

```
NameNode
DataNode
ResourceManager
NodeManager
```

---

10. Create an Input Directory in HDFS

Create an HDFS directory:

```
hdfs dfs -mkdir -p /matrix/input
```

---

11. Upload the Input File to HDFS

Upload the input file:

```
hdfs dfs -put matrix_input.txt /matrix/input/
```

Check that the file was uploaded:

```
hdfs dfs -ls /matrix/input
```

Display the input:

```
hdfs dfs -cat /matrix/input/matrix_input.txt
```

The output should be:

```
A,0,0,1
A,0,1,2
A,1,0,3
A,1,1,4
B,0,0,5
B,0,1,6
B,1,0,7
B,1,1,8
```

---

12. Execute the MapReduce Program

Run:

```
hadoop jar MatrixMultiplication.jar MatrixMultiplication /matrix/input
/matrix/output
```

The command has four important parts:

```
MatrixMultiplication.jar
```

This is the JAR file.

```
MatrixMultiplication
```

This is the main class.

```
/matrix/input
```

This is the HDFS input directory.

```
/matrix/output
```

This is the HDFS output directory.

---

13. Important: Output Directory Must Not Exist

Hadoop does not allow the output directory to already exist.

If you get an error such as:

```
FileAlreadyExistsException
```

remove the old output directory:

```
hdfs dfs -rm -r /matrix/output
```

Then execute the program again:

```
hadoop jar MatrixMultiplication.jar MatrixMultiplication /matrix/input
/matrix/output
```

---

14. View the Output

After the MapReduce job completes, run:

```
hdfs dfs -ls /matrix/output
```

You should see something similar to:

```
_SUCCESS
part-r-00000
```

Display the result:

```
hdfs dfs -cat /matrix/output/part-r-00000
```

Expected output:

```
0,0    19
0,1    22
1,0    43
1,1    50
```

Therefore, the final matrix is:

```
19  22
43  50
```

---

15. Matrix Multiplication Calculation

Matrix A:

```
1  2
3  4
```

Matrix B:

```
5  6
7  8
```

The first element is:

```
C[0][0] = (1 × 5) + (2 × 7)
        = 5 + 14
        = 19
```

The second element is:

```
C[0][1] = (1 × 6) + (2 × 8)
        = 6 + 16
        = 22
```

The third element is:

```
C[1][0] = (3 × 5) + (4 × 7)
```

```
        = 15 + 28
        = 43
```

The fourth element is:

```
C[1][1] = (3 × 6) + (4 × 8)
        = 18 + 32
        = 50
```

Final result:

```
19  22
43  50
```

---

16. How the Mapper Works

The Mapper reads each matrix element.

For example:

```
A,0,0,1
```

means:

```
Matrix = A
Row    = 0
Column = 0
Value  = 1
```

The Mapper sends the element to every possible output column.

For the example:

```
A,0,0,1
```

it produces:

```
(0,0) -> A,0,1
(0,1) -> A,0,1
```

Similarly:

```
B,0,0,5
```

produces:

```
(0,0) -> B,0,5
```

```
(1,0) -> B,0,5
```

The key represents the position of the result matrix.

For example:

```
0,0
```

means the element at row 0 and column 0 of the result.

---

17. How the Reducer Works

The Reducer receives all values belonging to the same output position.

For example, for:

```
0,0
```

the Reducer receives the required values from matrices A and B.

It separates them into:

```
aElements
bElements
```

Then it calculates the dot product:

```
A[0][0] × B[0][0]
+
A[0][1] × B[1][0]
```

which gives:

```
1 × 5 + 2 × 7
= 19
```

The Reducer finally writes:

```
0,0    19
```

---

18. Complete Command Sequence

If Hadoop is already installed and configured, the complete execution can be performed using these commands:

```
javac -classpath "$(hadoop classpath)" -d . MatrixMultiplication.java

jar -cvf MatrixMultiplication.jar *.class

start-dfs.sh

start-yarn.sh

hdfs dfs -mkdir -p /matrix/input

hdfs dfs -put matrix_input.txt /matrix/input/

hadoop jar MatrixMultiplication.jar MatrixMultiplication /matrix/input
/matrix/output

hdfs dfs -cat /matrix/output/part-r-00000
```

Expected result:

```
0,0    19
0,1    22
1,0    43
1,1    50
```

---

19. Short Viva Explanation

Question: What is the purpose of this program?

Answer:

This program performs matrix multiplication using Hadoop MapReduce. The Mapper generates intermediate key-value pairs for each possible output cell, and the Reducer calculates the dot product for each output cell.

Question: What is the Mapper output key?

Answer:

The Mapper output key represents the row and column of the result matrix, such as:

```
0,0
```

Question: What does the Mapper value contain?

Answer:

It contains the matrix identifier, common index, and matrix value, for example:

```
A,1,2
```

Question: What does the Reducer do?

Answer:

The Reducer separates the A and B elements, matches their common index, calculates their products, and adds the products to obtain the result.

Question: What is the output for the example?

Answer:

```
19  22
43  50
```

---

20. Important Note

The current program is configured specifically for a 2 × 2 example:

```
conf.set("m", "2");
conf.set("p", "2");
```

For larger or differently sized matrices, these dimensions must be changed appropriately.

The input format used by this program is:

```
Matrix,Row,Column,Value
```

For example:

```
A,0,1,2
```

means that matrix A has the value `2` at row `0`, column `1`.

---

21. Final File Structure

Your working directory can contain:

```
MatrixMultiplication/
│
├── MatrixMultiplication.java
├── MatrixMultiplication.jar
├── MatrixMultiplication.class
├── MatrixMultiplication$MatrixMapper.class
├── MatrixMultiplication$MatrixReducer.class
└── matrix_input.txt
```

The input and output directories are stored in HDFS:

```
HDFS
│
└── /matrix
    ├── input
    │   └── matrix_input.txt
    │
    └── output
        ├── _SUCCESS
        └── part-r-00000
```

Final Result

For:

```
A = 1  2
    3  4

B = 5  6
    7  8
```

the Hadoop MapReduce program produces:

```
A × B = 19  22
        43  50
```